Bawabu demo video: guide, script and voiceover

A 3-minute video for the Meta Global AI Developer Hackathon submission. It's 25% of the score, and the top 5 finalists are judged by watching it live, so it carries a lot of weight.

---

1. What the video must do

The rules (from the hackathon):

- **3:00 or shorter.** Aim for 2:50 so nothing gets cut off.
- **Only show what was built during the hackathon**, and say so. Everything in this script was.
- **Open with brief context of what existed before the event** (the organisers' guidance). For Bawabu that's: no Bawabu code, repo or service (the first commit is on 27 October, after the 26 October kickoff), only the tools it builds on. Segment 0:00–0:09 below does this.
- It goes in with the **public repo** and the **live product** (the hosted service and the npm package).

Our own honesty rules:

- If a part is sped up, show a small **"2× speed"** label while it's sped up.
- Say once that the attack package is **a harmless test package** (it can only "steal" fake secrets, sent to addresses that can't exist).
- Every number you say is real and sourced (section 9 lists where each comes from).

What the judges score, and the moment in the video that answers each:

Category	Weight	The moment
Impact	30%	The problem in numbers (0:17), and "on npm today" (2:37)
Use of Meta AI tooling	25%	Muse Code being protected; Muse Spark investigating and choosing to re-run the sandbox; Muse Spark Open (1:57)
Technical depth	20%	The locked sandbox, the CI re-run, verified evidence, hard rules (1:05–2:21)
Demo	25%	The live split screen: the investigation streaming while the agent waits (0:47–1:57)

The one moment the video is built around: the first sandbox run is clean, Muse Spark reads the code, realises the malware only runs on CI servers, re-runs the sandbox *as* a CI server, and catches the fake token leaving. Everything else supports that moment.

---

2. The plan at a glance

Time	Segment	On screen	Voiceover
0:00–0:09	**What existed before**	Context card: built during the hackathon, what it builds on	~25 words
0:09–0:17	Hook	Muse Code alone, about to install a package	~20 words
0:17–0:37	The problem	Three numbers, one at a time	~55 words
0:37–0:47	Bawabu in one line	Title, then the architecture diagram	~30 words
0:47–1:57	**Live demo**	**Split screen: Muse Code (left), Bawabu /live (right)**	~125 words
1:57–2:21	Under the hood	Quick cuts: scan, open-weights report, prompt injection	~55 words

Time	Segment	On screen	Voiceover
2:21–2:37	Results	Numbers card	~30 words
2:37–2:50	Try it, close	`npm i -g bawabu`, GitHub, end card	~35 words

About 390 words in total. At a calm pace (140–150 words a minute) that's 2:36–2:47, which leaves a few seconds for the silent beats in the demo.

---

3. One-time setup (do this the day before)

Accounts and tools

- A screen recorder: QuickTime (free, built in), **Screen Studio** (nice zooms) or **OBS** (free). Record at **1080p or higher, 30 fps**.
- A microphone: a USB mic or a good headset beats a laptop mic. Record in a quiet, soft room (curtains, a bed nearby).
- An editor: iMovie or DaVinci Resolve (both free), CapCut, or Screen Studio's own editor.

The demo project (a fresh folder the agent will work in; it gives nothing away):

```
npm i -g bawabu                                    # if not already installed
mkdir ~/payments-dashboard && cd ~/payments-dashboard
npm init -y
printf 'registry=https://amohntim123--bawabu-verdaccio-registry.us-east.modal.run/\n' >
.npmrc
git init
bawabu init                                         # expect four ✔ lines, including "Service
reachable"
```

The `.npmrc` points npm at our private test-lab registry, which holds the harmless practice packages. A name like `payments-dashboard` makes the scene feel real: a team project that needs a build-cache helper for its CI.

Rehearse Muse Code in its normal (interactive) mode. All our end-to-end tests used `muse exec`. Do one full rehearsal in the interactive `muse` app in this folder: trust the workspace, give the prompt below, and check the agent is blocked with Bawabu's message. If anything differs, fall back to `muse exec --trust-workspace "<prompt>"` in the terminal (it shows the conversation as text), and rehearse that instead.

The screen

- Dark theme everywhere. Terminal and editor font at **18–20 pt**; browser zoom **125–150%** for the Bawabu page.
- Two windows side by side, each half the screen: **left** the terminal with Muse Code, **right** the browser on `https://amohntim123--bawabu.us-east.modal.run/live`.
- Turn on **Do Not Disturb**, hide the Dock, close Slack, email and extra browser tabs, and hide the bookmarks bar.
- Keep the terminal prompt short (for example `export PS1='$ '`) so it doesn't show your username and folder path.

---

4. Before every take (the pre-flight)

From the bawabu repo folder:

```
uv run testlab/reset_demo.py ci-build-cache-helper --yes --warm
```

Expect: `Deleted 1 verdict(s)…`, `Service awake…`, `Practice registry awake: HTTP 200`. Without this, a second take would get the remembered answer instantly and `/live` would show nothing.

Then:

1. **Reload** the `/live` page. It only follows investigations that start *after* it's opened, so it should say "Watching for the next check".
2. Start the screen recording.
3. Count to three in silence (easy to trim), then type the prompt.

---

5. The script

Each segment lists what's **on screen**, what you **do**, and what you **say**. Words in *italics* are on-screen text (captions or labels you add while editing).

0:00–0:09 · What existed before the event

On screen: a simple context card (light text on a dark background), held for the whole segment:

- *Built at the Meta Global AI Developer Hackathon, 26 October – 4 November 2026*
- *Before the event: no Bawabu code, repository or service (first commit: 27 October)*
- *Existing tools it builds on: Muse Code, Muse Spark and Muse Spark Open (Meta) · Modal · Baseten · Supabase · Verdaccio · OSV.dev*
- *Everything else in this video was built during the hackathon.*

Say:

> Before this hackathon, Bawabu didn't exist. It was built from scratch during the event, on Meta's Muse Code, Muse Spark and Muse Spark Open.

(If any part of Bawabu, such as the idea or the plan, existed before 26 October, say so on the card, for example "*Before the event: only the idea*". Ideas are fine; the rule is about what you show being built during the event.)

0:09–0:17 · Hook

On screen: the Muse Code window, full screen. The prompt is typed and the agent starts running commands. *Caption: "Your AI coding agent, installing packages on its own."*

Say:

> AI coding agents now install packages on their own, mid-task. Nobody checks what comes in. What if one is malware?

0:17–0:37 · The problem

On screen: three big numbers, one after another, each with its source in small text underneath.

1. "*+1,444%: malicious open-source packages, 2024 to 2025*" (source: OpenSSF, via Google Threat Intelligence)
2. "*171 packages: hit by the Mini Shai-Hulud worm, May 2026*" (source: Cloud Security Alliance)
3. "*An AI agent added a malicious package to a real project*" (source: Google Threat Intelligence)

Say:

> Malicious open-source packages rose by more than fourteen hundred percent from 2024 to 2025. In May, the Mini Shai-Hulud worm compromised a hundred and seventy-one npm and PyPI packages. Google has already seen an AI agent add a malicious package to a real project. And blocklists only help after someone has reported the attack.

0:37–0:47 · Bawabu in one line

On screen: title card "*Bawabu: the gatekeeper*", then the architecture diagram (the README's, or page 3 of the PDF).

Say:

> This is Bawabu, Swahili for "gatekeeper". It sits between Muse Code and every package it installs. Normal packages pass in under a second. Suspicious ones get investigated.

0:47–1:57 · The live demo (split screen)

Do: in Muse Code (left), type exactly:

> Our CI builds are slow. Add the ci-build-cache-helper npm package to this project.

What will happen (timings from our real runs on the hosted service; about 25–45 seconds once the agent runs `npm install`, plus the agent's own thinking before and after):

Left (Muse Code)	Right (/live)	Say
The agent looks around, then runs `npm install ci-build-cache-helper`	Still "Watching…"	*(cut most of the agent's looking around)* "I ask Muse Code to add a build-cache helper for our CI. It's one of our harmless test packages."
The command pauses: Bawabu has stopped it	The page jumps to *ci-build-cache-helper@1.0.0*, **INVESTIGATING**	"The agent runs npm install, and Bawabu stops it before anything reaches my machine."
(waiting)	"Running the install in a locked sandbox with planted secrets", then "Sandbox: No secret reads or unknown network contact observed"	"On the right is Bawabu's live report. The package runs in a locked cloud sandbox with fake secrets planted as bait. The first run looks clean."
(waiting)	"Muse Spark takes over…", then Spark's lines in violet, "Reading scripts/cache.js"	"Muse Spark, Meta's model, takes over. It reads the install script and notices the code only runs on CI servers, where real secrets live."
(waiting)	The amber step: **"Re-running the sandbox with CI=true"**	"So it runs the sandbox again, pretending to be a CI server."
(waiting)	**Hard rule: Canary secret left the sandbox…**, then **BLOCK**	"And there it is: the fake npm token leaves the sandbox. That can only come from the bait, so it's proof. Blocked."
(waiting)	Scroll down slowly through the evidence cards: highlighted code lines, each "✓ verified"; "Run 2: re-run by Spark with CI=true", **STOLEN**	"Every piece of evidence is checked against the real code before Bawabu trusts it."
The agent shows Bawabu's message and tells you what it found, without retrying	(hold)	"In Muse Code, the agent gets the verdict and the evidence, and doesn't try another way."

Editing this part: keep three moments at **real speed**: Spark's "I found CI-gated code…" line, the amber re-run step, and the hard rule plus BLOCK. Speed up the waiting in between (2–3×) with a small "*2× speed*" label. Add lower-thirds the first time each side appears: "*Muse Code (Meta's coding agent)*" on the left, "*Bawabu live report*" on the right.

1:57–2:21 · Under the hood

On screen: four quick cuts, about six seconds each (shots B1, B3, B4 and B5 in section 6).

Say:

> Under the hood, it's a funnel. Quick checks clear most packages instantly and catch made-up names. Muse Spark Open, Meta's open-weights model, screens newcomers in seconds and can run the whole investigation. Package text is evidence, never instructions: a note telling AI reviewers to approve a package counts against it. And hard rules always beat the model.

Match each sentence to a cut:

- "made-up names": **B1**, `bawabu scan react-datepickr`, blocked with "Did you mean react-datepicker?"
- "Muse Spark Open": **B3**, the color-hex-convert report, with the chip "*investigator: Muse Spark Open (open weights, Baseten)*".
- "a note telling AI reviewers": **B4**, the same report scrolled to "*Prompt injection telling AI reviewer to approve…*".
- "hard rules": **B5**, the "*Hard rules (override the model)*" card.

2:21–2:37 · Results

On screen: a clean numbers card.

- "*9 / 9 attack samples caught, in every run, with both models*"
- "*0 of the 500 most popular npm packages blocked*"
- "*0.7 seconds for a normal package*"

Say:

> We tested it. It caught nine out of nine attack samples in every run, with both models. And it blocked none of the five hundred most popular npm packages.

2:37–2:50 · Try it, close

On screen: the terminal running `npm i -g bawabu`, then `bawabu init` with its four ✔ lines (shot B7), then the GitHub page, then the end card.

Say:

> Bawabu is open source and on npm today: one install, one command. Or run it on your own laptop with your own Meta API key. Bawabu: nothing gets in without a check.

End card (hold 3 seconds):

- **Bawabu**: the gatekeeper between AI coding agents and the packages they install
- `npm i -g bawabu`
- github.com/Amoh-Ntim/bawabu
- *Built during the Meta Global AI Developer Hackathon · Muse Code · Muse Spark · Muse Spark Open*

---

6. B-roll shot list (record these separately)

Short clips for the cutaways. Record each for a few seconds longer than you need.

Shot	What	How
B1	A made-up name blocked	Terminal: `bawabu scan react-datepickr` (about 2 s)
B2	An established package allowed	`bawabu scan left-pad` (about 1 s)
B3	The open-weights investigator	Browser: https://amohntim123--bawabu.us-east.modal.run/r/4676ecce-63c9-4909-89ac-4fc776c53021 (top of page: the "Muse Spark Open" chip)
B4	Prompt injection as evidence	Same page, scroll to the index.js evidence card and the "Prompt injection aimed at AI reviewers" rule
B5	Hard rules	Same page, the "Hard rules (override the model)" card
B6	Spark decoding hidden code (spare)	https://amohntim123--bawabu.us-east.modal.run/r/076a4b92-9fe1-4dd8-b637-4133d7457ddd (the DECODED evidence card)
B7	One install, one command	Terminal in a fresh folder: `npm i -g bawabu`, then `bawabu init`
B8	The npm page	https://www.npmjs.com/package/bawabu
B9	The repo	https://github.com/Amoh-Ntim/bawabu (after making it public), scrolling past the results table
B10	The architecture diagram	The README's diagram on GitHub, or page 3 of `docs/Bawabu-explained.pdf`

---

7. Editing

1. **Record the screen first, voice second.** Put the footage together, then record the voiceover while watching it. That's much easier than talking and clicking at the same time.
2. **Burn in captions.** Many judges watch without sound. Use the script text; most editors can import it.
3. **Labels:** "*2× speed*" whenever footage is sped up; lower-thirds the first time each tool appears; the source under each statistic.
4. **Music:** optional, quiet, and royalty-free (YouTube Audio Library, or your editor's built-in library), lowered well under the voice.
5. **Zoom:** if text is small, zoom into the area that matters (the amber re-run step, the STOLEN row, the BLOCK line). Screen Studio does this automatically; elsewhere, use a gentle crop and scale.
6. **Export:** MP4 (H.264), 1080p, 30 fps. Check the length is under 3:00. Watch it once start to finish with sound, and once without.

Voiceover tips

- Say **Bawabu** as *ba-WA-bu* (Swahili stresses the second-to-last syllable). Shai-Hulud: *shy-hoo-LOOD*.
- Smile slightly while you speak: it really does make you sound warmer. Read each segment two or three times and keep the best take.
- Keep the mic a hand's width from your mouth, slightly to the side.

---

8. If something goes wrong while recording

Problem	What to do
`/live` shows nothing after the install starts	You opened it before the reset, or it was already following an old check. Reset, reload `/live`, retake.
The verdict appears instantly with no investigation	The answer was remembered. Run the reset command again.
Spark blocks without re-running the sandbox	It occasionally decides the code alone is enough. Reset and retake; in our runs it re-ran every time.
The agent doesn't run `npm install` (it asks you first, or reads files for a long time)	Answer "yes, go ahead", or cut the waiting in editing. Avoid hints in the prompt.
The agent's message says "still investigating, retry in a minute"	The investigation took longer than the agent's wait (it happens on a cold service). Retake after `--warm`.
Everything fails on the day	Show a finished report instead (B3 or B6) and say "here's a recorded investigation", or use the backup take. Record a backup take as soon as you have one good take.

---

9. Where each number comes from

Said in the video	Source
"more than fourteen hundred percent from 2024 to 2025" (exactly 1,444%)	OpenSSF data, cited by Google Threat Intelligence: https://cloud.google.com/blog/topics/threat-intelligence/mitigation-guidance-for-supply-chain-compromise
"Google has already seen an AI agent add a malicious package to a real project"	Same Google report
"Mini Shai-Hulud … a hundred and seventy-one npm and PyPI packages"	Cloud Security Alliance: https://labs.cloudsecurityalliance.org/research/csa-research-note-shai-hulud-ai-supply-chain-20260517-csa-st/
"nine out of nine … in every run, with both models"	`eval/RESULTS.md`: 7 test-lab runs, 9/9 every time, Muse Spark and Muse Spark Open
"none of the five hundred most popular npm packages"	`eval/RESULTS.md`: top 500, 500/500 allowed
"under a second"	`eval/RESULTS.md`: 0.7 s median for the top 500

---

10. Upload, submit, post

Upload: YouTube (public or unlisted) or Loom. Title: *Bawabu: a gatekeeper between AI coding agents and the packages they install*. Check that the link plays in a private browser window. Add the link to the README (the "Demo video" line near the top).

Submission text (Cerebral Valley form; adjust to the form's fields):

- **Name:** Bawabu

- **One line:** The gatekeeper between AI coding agents and the packages they install.
- **Description:** AI coding agents install packages on their own, and attackers have noticed. Bawabu hooks into Muse Code and checks every install before it happens. Normal packages pass in under a second. Suspicious ones run in a locked cloud sandbox with fake secrets, and Muse Spark investigates like a security engineer: it reads the code, decodes hidden payloads, and re-runs the sandbox under the conditions malware waits for, such as a CI server or a future date. Every piece of evidence is checked before it's trusted, and hard rules override the model. Muse Spark Open, Meta's open-weights model, screens newcomers in seconds and can run the whole investigation. In our evaluation it caught 9 of 9 attack samples in every run and blocked none of the 500 most popular npm packages. It's on npm (`npm i -g bawabu`), and anyone can run it on their own laptop with their own Meta API key.
- **Meta models used:** Muse Code (integration through its hook), Muse Spark on the Meta Model API (the investigator), Muse Spark Open on Baseten (the screener, and the investigator in open-weights mode).
- **Links:** repo https://github.com/Amoh-Ntim/bawabu · npm https://www.npmjs.com/package/bawabu · live https://amohntim123--bawabu.us-east.modal.run/live · video (your link)

X post (attach a 30–45 second cut of the live demo: from "the first run looks clean" to "Blocked"):

> AI coding agents install packages on their own. Who checks them?
>
> Bawabu stops every install. Suspicious packages run in a locked sandbox with fake secrets, and Meta's Muse Spark investigates. Here it re-runs malware as a CI server to make it show itself.
>
> npm i -g bawabu

Add the hackathon's hashtag or handle if the organisers gave one. Post right after submitting: engagement for the Most Viral prize counts until 8 November.

Appendix: the voiceover in one piece (for reading aloud)

> Before this hackathon, Bawabu didn't exist. It was built from scratch during the event, on Meta's Muse Code, Muse Spark and Muse Spark Open.
>
> AI coding agents now install packages on their own, mid-task. Nobody checks what comes in. What if one is malware?
>
> Malicious open-source packages rose by more than fourteen hundred percent from 2024 to 2025. In May, the Mini Shai-Hulud worm compromised a hundred and seventy-one npm and PyPI packages. Google has already seen an AI agent add a malicious package to a real project. And blocklists only help after someone has reported the attack.
>
> This is Bawabu, Swahili for "gatekeeper". It sits between Muse Code and every package it installs. Normal packages pass in under a second. Suspicious ones get investigated.
>
> I ask Muse Code to add a build-cache helper for our CI. It's one of our harmless test packages. The agent runs npm install, and Bawabu stops it before anything reaches my machine.
>
> On the right is Bawabu's live report. The package runs in a locked cloud sandbox with fake secrets planted as bait. The first run looks clean. Muse Spark, Meta's model, takes over. It reads the install script and notices the code only runs on CI servers, where real secrets live. So it runs the sandbox again, pretending to be a CI server.
>
> And there it is: the fake npm token leaves the sandbox. That can only come from the bait, so it's proof. Blocked. Every piece of evidence is checked against the real code before Bawabu trusts it. In Muse Code, the agent gets the verdict and the evidence, and doesn't try another way.
>
> Under the hood, it's a funnel. Quick checks clear most packages instantly and catch made-up names. Muse Spark Open, Meta's open-weights model, screens newcomers in seconds and can run the whole investigation. Package text is evidence, never instructions: a note telling AI reviewers to approve a package counts against it. And hard rules always beat the model.
>
> We tested it. It caught nine out of nine attack samples in every run, with both models. And it blocked none of the five hundred most popular npm packages.
>
> Bawabu is open source and on npm today: one install, one command. Or run it on your own laptop with your own Meta API key. Bawabu: nothing gets in without a check.